

Sistemi Operativi - prova di laboratorio

Simulazione: Labirinto dei Draghi

Creare un programma **dragon-verifier.c** in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

```
./dragon-verifier <hero-points> <dragon-points> <quests>
```

Il programma deve verificare il punteggio finale di una spedizione dentro un labirinto. Ogni stanza contiene una mini-sfida tra un eroe e un drago. La sfida è descritta da tre file testuali.

File di input

I tre file forniti sulla riga di comando contengono, in ogni riga, rispettivamente:

- nel primo file, la potenza dell'eroe per la stanza corrente, come intero **long long**;
- nel secondo file, la potenza del drago per la stanza corrente, come intero **long long**;
- nel terzo file, l'azione da applicare: **L**, **C** oppure **B**. L'ultima riga del terzo file contiene invece il risultato finale atteso.

Le azioni hanno il seguente significato:

Azione	Nome	Calcolo della stanza
L	Loot	eroe + drago
C	Combattimento	eroe - drago
B	Boss fight	eroe x drago

Il punteggio finale è la somma semplice dei risultati di tutte le stanze:

$$totale = r_1 + r_2 + r_3 + \dots + r_n$$

Thread richiesti

Al suo avvio il programma deve creare esattamente 4 thread ausiliari oltre al thread principale:

- un thread **HERO**, che legge le potenze dell'eroe dal primo file;
- un thread **DRAGO**, che legge le potenze del drago dal secondo file;
- un thread **QUEST**, che legge le azioni dal terzo file, mantiene localmente la sommatoria e verifica il risultato finale;
- un thread **MOTORE**, che calcola il risultato della stanza quando potenza eroe, potenza drago e azione sono disponibili.

Vincoli obbligatori

- Usare esattamente **due variabili condizione** e **un mutex**.
- Non usare semafori, pipe, sleep per sincronizzare, busy waiting o barriere pthread.
- Non usare strutture dati condivise con visibilità globale.
- L'unica struttura dati condivisa deve essere passata ai thread tramite argomento.
- Tutti i thread devono terminare spontaneamente alla fine dei lavori.
- Il codice deve compilare con gcc o clang usando le opzioni indicate sotto.

Struttura condivisa suggerita

La struttura può essere estesa con altri flag, ma deve rispettare il vincolo di un solo mutex e due condition variable:

```
typedef struct {  
    pthread_mutex_t mutex;  
    pthread_cond_t cond_motore;  
    pthread_cond_t cond_sync;  
  
    long long potenza_eroe;
```

```
    long long potenza_drago;
    char azione;
    long long risultato;

    int disponibile_eroe;
    int disponibile_drago;
    int disponibile_azione;
    int risultato_disponibile;

    int fine_eroe;
    int fine_drago;
    int fine_quest;
    int fine_lavori;

    long long indice_stanza;
} shared_t;
```

Comportamento richiesto

Per ogni stanza del labirinto:

- HERO legge una potenza dal primo file e la deposita nella struttura condivisa;
- DRAGO legge una potenza dal secondo file e la deposita nella struttura condivisa;
- QUEST legge l'azione dal terzo file e la deposita nella struttura condivisa;
- MOTORE attende che i tre dati siano disponibili, calcola il risultato della stanza e lo deposita nella struttura condivisa;
- QUEST attende il risultato, aggiorna una propria variabile locale **totale** e stampa il totale parziale.

Quando QUEST incontra l'ultima riga del file delle azioni, interpreta quella riga come risultato finale atteso. A quel punto confronta il valore atteso con il totale calcolato e stampa se la spedizione è riuscita o fallita.

Formato dell'output

È necessario rispettare il formato delle righe mostrato negli esempi. L'ordine delle prime stampe di avvio dei thread può dipendere dallo scheduling, ma la sequenza delle stanze deve essere coerente.

Compilazione

```
gcc -Wall -Wextra -Werror -pthread dragon-verifier.c -o dragon-verifier
```

Esecuzione

```
./dragon-verifier A.hero A.dragon A.quest  
./dragon-verifier B.hero B.dragon B.quest
```

File di esempio A

```
A.hero  
10  
5  
8  
3  
12  
7  
20  
4
```

```
A.dragon  
4  
9  
2  
6  
3  
7  
5  
11
```

```
A.quest  
L  
C  
B  
C  
L  
B  
C  
L  
117
```

Output atteso su esempio A

```
$ ./dragon-verifier A.hero A.dragon A.quest  
[MAIN] creo i thread ausiliari  
[HERO] leggo le potenze dal file 'A.hero'  
[DRAGO] leggo le potenze dal file 'A.dragon'  
[QUEST] leggo le missioni e il risultato atteso dal file 'A.quest'  
[HERO] potenza eroe n.1: 10
```

```
[DRAGO] potenza drago n.1: 4
[QUEST] azione n.1: L
[MOTORE] stanza n.1: 10 L 4 = 14
[QUEST] punteggio dopo 1 stanza/e: 14
[HERO] potenza eroe n.2: 5
[DRAGO] potenza drago n.2: 9
[QUEST] azione n.2: C
[MOTORE] stanza n.2: 5 C 9 = -4
[QUEST] punteggio dopo 2 stanza/e: 10
...
[HERO] potenza eroe n.8: 4
[HERO] termino
[DRAGO] potenza drago n.8: 11
[DRAGO] termino
[QUEST] azione n.8: L
[MOTORE] stanza n.8: 4 L 11 = 15
[MOTORE] termino
[QUEST] punteggio dopo 8 stanza/e: 117
[QUEST] risultato finale atteso: 117 (spedizione riuscita)
[QUEST] termino
[MAIN] termino il processo
```

File di esempio B

B.hero

-2
15
0
6
9

B.dragon

5
-3
10
6
2

B.quest

L
C
B
B
C
64

Output atteso su esempio B

```
$ ./dragon-verifier B.hero B.dragon B.quest
[MAIN] creo i thread ausiliari
[HERO] leggo le potenze dal file 'B.hero'
[DRAGO] leggo le potenze dal file 'B.dragon'
[QUEST] leggo le missioni e il risultato atteso dal file 'B.quest'
[HERO] potenza eroe n.1: -2
[DRAGO] potenza drago n.1: 5
[QUEST] azione n.1: L
[MOTORE] stanza n.1: -2 L 5 = 3
[QUEST] punteggio dopo 1 stanza/e: 3
[HERO] potenza eroe n.2: 15
[DRAGO] potenza drago n.2: -3
[QUEST] azione n.2: C
[MOTORE] stanza n.2: 15 C -3 = 18
[QUEST] punteggio dopo 2 stanza/e: 21
[HERO] potenza eroe n.3: 0
[DRAGO] potenza drago n.3: 10
[QUEST] azione n.3: B
[MOTORE] stanza n.3: 0 B 10 = 0
[QUEST] punteggio dopo 3 stanza/e: 21
[HERO] potenza eroe n.4: 6
[DRAGO] potenza drago n.4: 6
[QUEST] azione n.4: B
[MOTORE] stanza n.4: 6 B 6 = 36
[QUEST] punteggio dopo 4 stanza/e: 57
[HERO] potenza eroe n.5: 9
[HERO] termino
[DRAGO] potenza drago n.5: 2
[DRAGO] termino
[QUEST] azione n.5: C
[MOTORE] stanza n.5: 9 C 2 = 7
[MOTORE] termino
[QUEST] punteggio dopo 5 stanza/e: 64
[QUEST] risultato finale atteso: 64 (spedizione riuscita)
[QUEST] termino
[MAIN] termino il processo
```

Casi di errore da gestire

- Numero errato di argomenti: stampare un messaggio d'uso su stderr e terminare con codice di errore.
- File non apribile: stampare un errore con perror e terminare il processo.
- Azione non valida nel file delle missioni: trattarla come errore di input.
- Numero diverso di righe nei file degli eroi e dei draghi: il programma non deve bloccarsi.

Checklist per la correzione

- Il programma crea 4 thread ausiliari e il main li attende tutti.
- Sono presenti esattamente due pthread_cond_t e un pthread_mutex_t nella struttura condivisa.

- La sommatoria è mantenuta localmente dal thread QUEST, non dal main.
- Non sono presenti variabili globali condivise.
- Tutti i pthread_cond_wait sono protetti da cicli while.
- Il programma termina senza deadlock anche con molti elementi nei file.

Tempo consigliato: 1 ora e 35 minuti.